CSC 330: Assignment 5

Deadline: **Thu, Apr 2** at **23.59**

Submit via: CSC 330 Submission Portal

Ground rules

- You must complete this assignment *entirely* on your own. In other words, you should come up with the solution *yourself*, write the code *yourself*, and test the code *yourself*. The university policies on academic dishonesty (a.k.a. cheating) will be taken very seriously. Check the syllabus for details on what is allowed and what is not.
  - You can have high-level discussions with your classmates about the course material.
  - You are also welcome to use Teams or come to office hours and ask questions. If in doubt, please ask—we are here to help!
  - Using LLMs to get the solutions is akin to shooting yourself in the foot: you *might* pass the assignment, but you will most likely fail the written exams.
- You can use a maximum of **three (3) late days per assignment**, and a maximum of **seven (7) days for the whole semester**. Any further accommodations must be announced and negotiated with the instructor at least seven (7) days before the deadline.
- You will need to gather at least half of the total assignment score to pass the course.
- If you think some things should be explained in detail, please include the explanations in the form of source code comments.
- We will try to grade your assignments within seven (7) days of the initial submission deadline.
- If you think there is a problem with your grade, you have one week to raise concerns after the grades go public. TAs will be holding office hours during those seven days to address any such problems. After that, your grade is set in stone.
- Document your code where needed!
- Your programs should generate **no** errors or warnings. If a program generates a single compilation error or warning, it might receive a zero.
- Please indicate your V-number and name within a comment at the top of each submitted file.
  - **Do not** include other private information (full name, your address, telephone number, medical exam tests etc.).

Furthermore, for this assignment:

- You should only submit a single ZIP file containing two a single APL and a single Ruby file to the CSC 330 Submission Portal. The contents of the ZIP must include `prob1.rb` and `prob2.apl` (lowercase).
  - Warning: different file names might result in zero points. Our scripts expect the correct file names.
  - Another warning: put the filenames at the root of the ZIP file; if you put them within a directory, our script might not be able to find it.
- **Each APL function must fit one line!** So, for each APL task, you only need to write a single line!
- You can use anything in APL except for ⎕-prefix functions (`⎕io←0` is allowed, though).
- APL test cases are given below. That's all there is.

Test examples, updates and error corrections will be available on Teams and the CSC 330 Submission Portal. Make sure to check it from time to time.

Problem 1: Fumix [15 points]

Lists are versatile data structures, and almost no implementation is as comprehensive as Ruby's `Array`! Let's use those arrays to implement a data structure for polymorphic **functional** sets (hello again, Assignment 2). To recap: a functional set can be defined as a array of (1) *ordered*, and (2) *unique* elements, where the order is maintained by some comparator function f. For this particular problem, the set order is defined by a function `cmp` that takes two elements and returns:

- a zero if the elements are equal,
- a positive integer if the first element is greater than the second, and
- a negative integer if the first element is smaller than the second one.

Now consider the following mixin:

```ruby
module FuncSet
  def |(other)  # union
    self.class.new(entries + other.entries) { |x, y| cmp(x, y) }
  end
  def &(other)  # intersect
    common = self.map { |x| other.find { |y| self.cmp(x, y) == 0 } }
    self.class.new(common) { |x, y| cmp(x, y) }
  end
end
```

Implement the class `MySet` with the following functions:

1. [3 points] `initialize` that takes an optional array of items and the comparison function as a block. If no block is passed, use the default spaceship operator `<=>`.
2. [3 points] `to_s` that returns a string representation of the set in array format (see the example below), and `cmp` that uses the comparator to compare the elements.

   Warning: `puts` is weird with arrays. Make sure that `to_s` returns a string for everything to work.
3. [3 points] `insert` that inserts an element into the set. If the element is `nil` do nothing.

Then:

4. [3 points] Implement the `Enumerable` mixin. In order for it to work, you need to provide `each` method that has the same behaviour as `Array.each`.
5. [3 points] Implement the `FuncSet` mixin.

You are not allowed to subclass `Array`. You can use `Array` objects internally, though.

Bonus

6. [2 points] Is `map` that we got for free from `Enumerable` safe to use? Why, or why not?

Example

```ruby
f = MySet.new([1, 2, 3, 2, 1.5])
puts f
# [1, 1.5, 2, 3]
f.each { |i| puts "- #{i}" }
# - 1
# - 1.5
# - 2
# - 3
print f.map { |x| x.round % 2 }, "\n"
# [1, 0, 0, 1]
puts f.count  # 4
puts f.class  # MySet

puts f | MySet.new([1, -1])
# [-1, 1, 1.5, 2, 3]
puts f & MySet.new([1, 2, 5])
# [1, 2]
```

```ruby
puts f
# [1, 1.5, 2, 3]

f = MySet.new([1, 2, 3, 2, 1.5]) { |a,b| -(a<=>b) }
puts f
# [3, 2, 1.5, 1]
```

Problem 2: Deoxyribonucleic Arrays [10 points]

A DNA (deoxyribonucleic acid) string is composed of the letters 'A','C','G' and 'T'.

1. [6 points] Write an APL function `count` in the direct (dfn) style that counts the number of each DNA nucleotide in a given string.
2. [4 points] Write a tacit function `count_t` that matches the behaviour of `count`.

The GC content of a DNA string is given by the percentage of the symbols in the string that are either 'C' or 'G'. Small discrepancies in GC content can be used to distinguish species: for example, most bacteria have a GC content significantly higher than 50%.

3. [6 points] Write an APL function `gc` in the direct (dfn) style that returns GC content of the string as a percentage.
4. [4 points] Write a tacit function `gc_t` that matches the behaviour of `gc`.

And finally… a magic square is a square array of numbers if the sums of the numbers in each row, each column, and (both) main diagonals are the same.

5. [10 points] Write a direct (dfn) function `magic` to test whether an array is a magic square. The function returns 1 if the array is a square, and 0 if not. Assume that the argument rank is always 2 and that the matrix is square (in APL terms, `1 = 2=/⍴X` for an array `X`).
6. [5 points] Write a tacit function `magic_t` that matches the behaviour of `magic`.

Example

```
    count 'AGCTTTTCATTCTGACTGCAACGGGCAATATGTCTCTGTGTGGATTAAAAAAAGAGTGTCTGATAGCAGC'
20 12 17 21
    count ''
0 0 0 0
    count 'HELLO'
0 0 0 0
    count_t 'G'
0 0 1 0

    gc 'GCGCGCGCCCGGGGCCG'
100
    gc 'HELLO'
0
    gc_t 5 10 10 5 / 'ACGT'
66.66666667

    magic 1 1⍴42
1
    magic_t 3 3⍴4 9 2 3 5 7 8 1 6
1
    magic 2 2⍴1 2 3 4
0
```

Hints

- You might find `∊` (membership) and `⍨` (commute) useful!
- Outer products are often hiding in plain sight!
- You might find `[]` (squad indexing) and `⍉` (transpose) useful!
- Remember: tacit functions should not contain `⍺` or `⍵` (otherwise, you will get 0 points).

(see next page)

Notes

Your APL file should look like this:

```
⎕io ← 0
count ← { 0 }    ⍝ replace 0 with your one-line implementation
count_t ← ( ⊢ )  ⍝ replace ⊢ with your one-line implementation
gc ← { 0 }       ⍝ replace 0 with your one-line implementation
gc_t ← ( ⊢ )     ⍝ replace ⊢ with your one-line implementation
magic ← { 0 }    ⍝ replace 0 with your one-line implementation
magic_t ← ( ⊢ )  ⍝ replace ⊢ with your one-line implementation
```

If it does not look like this, you might fail all test cases.

Sumbit **a text file** with `.apl` extension. Easiest way to do that is to open a text editor (e.g., Sublime, VS Code or vim) and copy/paste the completed function from Dyalog or TryAPL there. Please **do not** submit Dyalog or TryAPL workspace.